ЛАБОРАТОРНАЯ РАБОТА №2

# Оптимизация доступа в кэш

Блочное умножение матриц из лабораторной №1 на Apple M5

> Работа берёт векторизованное ядро из лабораторной №1 и, **сохраняя матрицу матриц**, оптимизирует доступ к памяти под иерархию кэшей. Подматрица B держится в L2, блоки A и столбец подматрицы B — в L1, пять строк блока результата — в регистрах. Размеры подматриц на обоих уровнях даёт формула задания.
>
> Все версии совпадают **побитово** на случайных дробных данных. На одном ядре C2 быстрее C1 в **1,56–1,71 раза** и скалярной версии — в **2,32–2,61 раза**; с FMA — в 2,14 и 2,83 раза. Вклад регистров и подматриц измерен отдельно.

МАТРИЦА

384 × 384 блока по 10 × 10 double

ПЛОСКИЙ РАЗМЕР

3840 × 3840, mod 64 = 0

ПОДМАТРИЦА B

48 × 128 блоков, 4,9 МБ, ширины кратны 64

ДАННЫЕ

случайные double из [−1, 1)

ЦЕЛЕВАЯ МАШИНА

Apple M5, ARM64, macOS, одно ядро

SIMD

SSE2-интринсики → NEON (sse2neon)

# Содержание


1 Что требуется — задание и ограничения

2 Целевая машина — кэши M5 и цена промаха

3 Блоки, C0 и C1 — раскладка в памяти, скалярная база, версия только с векторизацией

4 Размер подматриц по формуле задания — L/3 · 0,9 на двух уровнях, кратность 64 байтам

5 C2: оптимизация под кэш — подматрицы в L2 и L1, регистры

6 Побитовая идентичность — почему результат не меняется и почему данные случайные

7 Результаты измерений — тайминги и инструкции

8 Анализ — вклад регистров и подматриц, сравнение с лаб. №1

9 Вопросы на защиту — короткие ответы

# 1 Что требуется

> Взять векторизованную версию из лабораторной №1 и оптимизировать её под кэш, сохранив матрицу матриц. Доказать, что результат не изменился, и обосновать преимущества оптимизации.

| ТРЕБОВАНИЕ | КАК ВЫПОЛНЕНО |
|---|---|
| Исходник — версия **только с векторизацией** из лаб. №1 | C1 — ядро SSE2 из лабораторной №1 без изменений, на той же блочной матрице, блоки в исходном порядке i → j → k — без какой-либо оптимизации под кэш (§3) |
| Матрица матриц сохраняется | строка «матрицу матриц можно удалить» в задании **зачёркнута**; A, B и C — сетки 384 × 384 блоков по 10 × 10 чисел, раскладка как в лаб. №1 |
| Две версии: C1 векторизованная, C2 по заданию | `multiply_vectorized` и `multiply_blocked`; для сравнения добавлена скалярная C0 — `multiply_scalar` |
| C1 и C2 совпадают **полностью** | `memcmp` по всему массиву, без epsilon, на случайных дробных данных; от проверки зависит код возврата |
| Обосновать преимущества оптимизации | замер задержек `make probe` (§2), раздельный замер вклада регистров и подматриц, трафик к памяти (§8) |
| mod(size, 64) = 0 | 3840 = 384 · 10, кратно 64; проверяется `_Static_assert` |
| blocksize = $\lfloor L3_{size} / 3 \cdot 0{,}9 \rfloor$ — максимальный размер одного блока | в задании эта единица названа блоком; здесь блок — это 10 × 10 из лаб. №1, поэтому она названа **подматрицей**. Формула вычисляется во время выполнения и применена к L2 и к L1d (§4) |
| Ширина подматриц A и B в байтах кратна 64 | 48 блоков = 3840 байт и 128 блоков = 10 240 байт; ширины всегда кратны 8 блокам = 640 байт |
| Блок одной матрицы неизменен, пока не обработаны все блоки второй | подматрица B держится, пока мимо неё проходят все 384 строки блоков A |
| Замер: high_resolution_clock или rdtsc | `CLOCK_MONOTONIC_RAW` и счётчик `cntvct_el0` — аналог rdtsc на ARM; оба печатаются |
| Запрет `clock()`, `time()`; только Release | соблюдено; сборка `-O3` |
| Запрет транспонирования из лаб. №1 | соблюдён: ничего не копируется и не переставляется, блоки B читаются на месте |

### Бонусные пункты

| БОНУС | СТАТУС |
|---|---|
| Оптимизация не только под L3, но и под L2, L1 | У M5 два уровня кэша: L1 на ядро и L2 на кластер, играющий роль L3. Формула задания применена к обоим: L1d задаёт глубину подматрицы, L2 — её ширину; плюс регистры (§4, §5) |
| Расширенная версия по cpumemory.pdf | По §6.2.1 Drepper'a: подзадачи, помещающиеся в кэш, и переиспользование загруженного — на трёх уровнях (L2, L1, регистры). Почему C1 медленная — по §6.3.1 (предвыборка), подтверждено замером `make probe` |
| Intel vTune | Недоступен на Apple silicon. Аналог — Instruments → CPU Counters, требует Xcode |

## 2 Целевая машина

Параметры кэшей программа читает во время выполнения через `sysctl`:

| УРОВЕНЬ | РАЗМЕР | ПРИМЕЧАНИЕ |
|---|---|---|
| L1d (P-ядра) | `128 KiB` | на ядро производительного кластера |
| L1d (E-ядра) | `64 KiB` | экономичный кластер |
| L2 (P-кластер) | `16 MiB` | общий на 4 производительных ядра |
| L2 (E-кластер) | `6 MiB` | общий на 6 экономичных ядер |
| **L3** | `отсутствует` | у Apple silicon нет архитектурного L3 |
| Кэш-линия | `128 байт` | вдвое больше обычных 64 байт на x86 |
| Страница памяти | `16 KiB` | вчетверо больше обычных 4 KiB на x86 |

**ДВЕ ЛОВУШКИ**

**Первая.** `sysctl hw.l1dcachesize` и `hw.l2cachesize` возвращают 64 KiB и 6 MiB — значения *экономичного* кластера. Код читает `hw.perflevel0.*`, иначе расчёт подматриц занижен.

**Вторая.** Архитектурного L3 нет. Его роль в формуле задания берёт L2 производительного кластера: на x86 L3 — общий кэш ядер, а у M5 общий для кластера кэш — это L2. Программа печатает это явно.

Программа однопоточная. Запущенная из терминала, она получает QoS user-interactive и выполняется на производительном ядре. В фоновом режиме процесс прижимается к экономичным ядрам, под кэши которых подматрицы не рассчитаны, — программа определяет это и печатает предупреждение. Режим низкого энергопотребления снижает частоту примерно вдвое; все замеры сняты с ним выключенным.

### Цена промаха на этой машине: `make probe`

Отдельная программа `cache_probe` измеряет задержку одной загрузки в зависимости от размера рабочего множества. В буфере лежит по одному указателю на кэш-линию, программа идёт по цепочке, и каждая загрузка зависит от предыдущей — поэтому видна полная задержка. Цепочка строится двумя способами: в **случайном** порядке, который не угадать никакой предвыборке, и в **порядке адресов**, который аппаратная предвыборка распознаёт.

| РАБОЧЕЕ МНОЖЕСТВО | СЛУЧАЙНО, НС | ПОДРЯД, НС | ГДЕ ЛЕЖИТ |
|---|---|---|---|
| 16–128 KiB | 0,68 | 0,68 | L1d |
| 256–512 KiB | 2,7–2,8 | 1,3 | L2 |
| 2–8 MiB | 7,0–8,4 | 1,7–1,8 | L2, промахи TLB |
| 16 MiB | 17,3 | 2,4 | на пределе L2 |
| **64–512 MiB** | **82–86** | **6,6–6,7** | **память** |

**Размер рабочего множества**

Загрузка из памяти стоит в **127 раз** дороже, чем из L1, и в 10–30 раз дороже, чем из L2. Отсюда правило: то, что переиспользуется, держать в кэше. Это делают подматрицы.

**Порядок обращений**

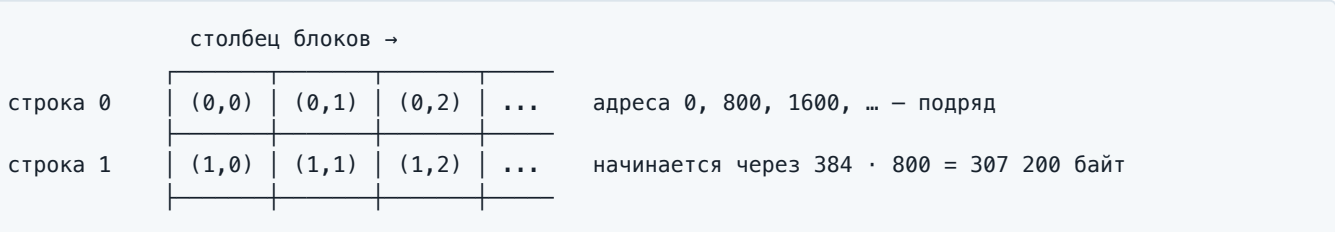
Из памяти подряд — 6,6 нс, вразброс — 86: предвыборка скрывает **92 %** задержки, но только при последовательном чтении. C1 читает B вразброс (§3).

Числа между прогонами немного плавают (в диапазоне 1–32 MiB — до 1,6 раза), ступени на 128 KiB и 16 MiB совпадают с `sysctl` всегда.

## 3 Блоки, C0 и C1

### Как лежат блоки

Каждая матрица — сетка 384 × 384 блока, блок — матрица 10 × 10 чисел `double`, по строкам. Блок — это **800 байт подряд**, 6,25 кэш-линии. Блоки лежат строка за строкой: блок (r, c) начинается с элемента `(r · 384 + c) · 100`.




Отсюда два направления: **вправо по строке блоков** следующий блок лежит вплотную, а **вниз по столбцу блоков** — на 307 200 байт дальше, на другой странице памяти. Для блока результата `C[i][j] = Σ A[i][k] · B[k][j]` нужна строка блоков A (подряд) и столбец блоков B (с прыжками); каждое «·» — произведение двух блоков 10 × 10, 2000 операций.

### C1: только векторизация

C1 — это ядро SSE2 из лабораторной №1 без единого изменения. Блоки обходятся в исходном, «учебном» порядке i → j → k: каждый блок результата получает всю свою сумму, и только потом начинается следующий.

```
for (block_row)                                  // строка блоков C
    for (block_col)                              // блок результата
        for (block_k)                            // вся его сумма по k
            C[block_row][block_col] += A[block_row][block_k] · B[block_k][block_col];
```

Произведение пары блоков — схема Intel AP-930: элемент A размножается по дорожкам вектора и умножается на строку блока B, которая читается подряд. Строка блока результата — 10 чисел, пять регистров `__m128d` — живёт в регистрах на время одного произведения блоков.

> **ЧЕМ C1 ОТЛИЧАЕТСЯ ОТ ПРОГРАММЫ ЛАБОРАТОРНОЙ №1**
>
> В лабораторной №1 блоки шли в порядке i → k → j. Этот порядок — уже оптимизация под кэш: при нём строка блоков B читается подряд, и работает предвыборка. Задание требует, чтобы C1 была версией *только* с векторизацией, поэтому здесь ядро то же, а порядок блоков исходный. C2 решает ту же проблему другим способом — подматрицами в кэше (§5).
>
> Поэтому C1 здесь медленнее, чем SSE2-версия в выводе лабораторной №1: 17,9–21,4 против 31,2 GFLOP/s. Сравнение двух лабораторных — в §8.

### C0: скалярная база

Чтобы было видно, что даёт каждая из двух лабораторных, программа замеряет и скалярную версию. C0 — простое ядро лабораторной №1 ( `inner_row` → `inner_k` → `inner_col` ), собранное с `-fno-vectorize -fno-slp-vectorize` , в том же порядке блоков i → j → k, что и C1. Поэтому C1 / C0 — эффект векторизации и ничего больше, C2 / C1 — эффект оптимизации под кэш, C2 / C0 — обеих лабораторных вместе. `make disasm` показывает, что в C0 нет ни одной векторной арифметической инструкции.

### Что блочная раскладка уже сделала

Каждый блок — 800 непрерывных байт, поэтому даже в порядке i → j → k любая подтянутая кэш-линия используется целиком. На плоской матрице C1 с вектором по j читала бы B по столбцам и брала бы из каждой 128-байтной линии только 16 байт. Именно так получаются разрывы на порядок в работах с плоскими матрицами. Блочная раскладка убрала эту проблему ещё в лабораторной №1.

### Что осталось

- **Столбец B читается с прыжками.** Соседние блоки столбца лежат в 307 КБ друг от друга, на разных страницах; по мере движения по j это 384 переплетённых потока. Drepper (§6.3.1) пишет, что аппаратная предвыборка отслеживает от восьми до шестнадцати потоков и не пересекает границ страниц, — каждый блок B приходится ждать.
- **B читается из памяти 384 раза.** Вся матрица B (118 МБ) не помещается в кэш и заново читается для каждой строки блоков C — около 45,7 ГБ.

- **С гоняется через память.** Строка блока результата загружается и сохраняется на каждое произведение блоков — 384 раза за сумму.

## 4 Размер подматриц по формуле задания

$$block_{size} = \lfloor\, L_{size} / 3 \cdot 0{,}9 \,\rfloor$$

**Подматрица** — прямоугольник из целых блоков, который держится в кэше, пока с ним работают. В задании эта единица названа блоком; здесь блок — это 10 × 10 из лабораторной №1, поэтому прямоугольник из блоков называется подматрицей.

В уровне кэша одновременно должны уместиться три подматрицы — от A, от B и от C, — отсюда деление на три. Множитель 0,9 оставляет десятую часть на остальное. Код считает формулу как `L · 90 / 300` одним целочисленным делением: вариант «сначала поделить на 3» округляет дважды и для некоторых L ошибается на единицу. Формула применяется к двум уровням.

| УРОВЕНЬ | L | ⌊L/3 · 0,9⌋ | ЧТО ОГРАНИЧИВАЕТ | ИТОГ |
|---|---|---|---|---|
| **L1d** | 128 KiB | 39 321 Б | блоки строки A, которые встречаются с подматрицей B (1 × глубина), и столбец подматрицы B (глубина × 1) — каждый ≤ 39 321 байт | глубина 48 |
| **L2 (роль L3)** | 16 MiB | 5 033 164 Б | вся подматрица B, глубина × ширина блоков | ширина 128 |

39 321 байт вмещают 49 блоков по 800 байт; из делителей 384, кратных 8, наибольший не больше 49 — **48**. При глубине 48 бюджет L2 вмещает 131 блок в ширину; подходящий делитель — **128**. Подматрица B — 48 × 128 блоков = 4,9 МБ при разрешённых 5,03 МБ; матрица B режется на 8 × 3 = 24 такие подматрицы.

```
=== cache geometry ===
cache line              : 128 bytes
L1 data                 : 128 KiB
L2                      : 16384 KiB
L3                      : absent, L2 takes its place
max sub-matrix, L2      : 5033164 bytes (L2 / 3 * 0.9)
max sub-matrix, L1d     : 39321 bytes (L1d / 3 * 0.9)
B sub-matrix, in L2     : 48 x 128 blocks, 4915200 bytes
  width of A sub-matrix : 48 blocks = 3840 bytes (mod 64 = 0)
  width of B sub-matrix : 128 blocks = 10240 bytes (mod 64 = 0)
in L1d                  : A 1 x 48 blocks + B column 48 x 1 blocks, 38400 bytes each
in registers            : 5 rows x 10 columns of a result block (25 accumulators)
```

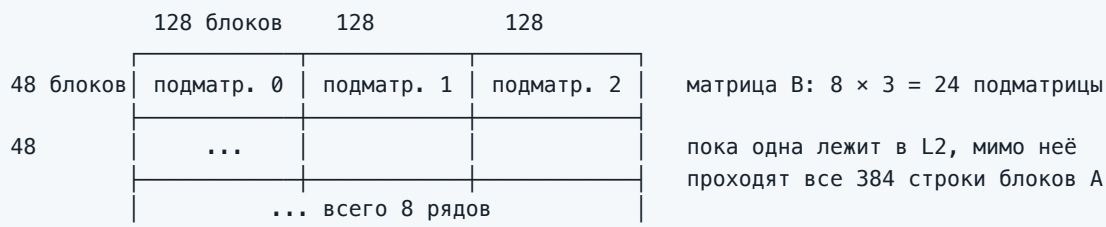
### Кратность ширины 64 байтам — в блочной матрице

Подматрица шириной *w* блоков имеет ширину `w · 10 · 8 = 80·w` байт как матрица и занимает `w · 800` подряд идущих байт на каждую строку блоков в памяти. При *w*, кратном 8, оба числа кратны и 64 байтам (требование задания), и 128 — кэш-линии M5: каждая строка блоков подматрицы начинается на границе линии и занимает целое число линий. Поэтому ширины подматриц в коде всегда кратны 8 блокам.

Форма подматрицы на скорость почти не влияет: при разработке 64 × 64, 48 × 48, 96 × 96 и 48 × 128 блоков дали одинаковую скорость в пределах 3 %. Выбрана та, что следует из формулы на обоих уровнях. Переменные окружения `LAB2_DEPTH` и `LAB2_WIDTH` позволяют задать размеры вручную для экспериментов.

## 5 C2: оптимизация под кэш

C2 выполняет ту же арифметику, что C1, и в том же порядке i → j → k — но не по всей матрице, а внутри подматрицы B, которая лежит в кэше. Идея: взять кусок B, который помещается в кэш, загрузить его один раз и сделать **всю** работу, которой он нужен, прежде чем перейти к следующему.




| УРОВЕНЬ | ЧТО ТАМ ДЕРЖИТСЯ | РАЗМЕР |
|---|---|---|
| **L2 (роль L3)** | подматрица B: 48 × 128 блоков. Неизменна, пока мимо неё проходят все 384 строки блоков A. Спуск по её столбцам идёт с прыжками, но попадает в L2, а не в память | 4,9 МБ |
| **L1d** | 48 блоков строки A — они лежат подряд и нужны для всех 128 столбцов подматрицы; столбец подматрицы B (48 блоков) — нужен обеим половинам блока результата | 2 × 38,4 КБ |
| **Регистры** | 5 строк блока результата — 25 аккумуляторов на всю глубину подматрицы, 48 произведений блоков подряд | 5 × 10 |

```
for (first_k  = 0; first_k  < 384; first_k  += 48)          // по глубине — по возрастанию
  for (first_col = 0; first_col < 384; first_col += 128)      // подматрица B неизменна...
    for (block_row = 0; block_row < 384; ++block_row)        // ...пока идут ВСЕ строки A
      for (block_col in подматрице)                          // 128 столбцов
        for (половина = строки 0–4, затем 5–9) {
          acc[5][5] = C[block_row][block_col], строки половины;  // 25 регистров
          for (block_k in подматрице)                        // 48 блоков столбца B — из L2
            for (inner_k = 0; inner_k < 10; ++inner_k)
              acc += A[block_row][block_k] · B[block_k][block_col];
          C[block_row][block_col] = acc;
        }
```

Это ровно правило задания: «блок одной из матриц остаётся без изменений до тех пор, пока не будут обработаны ВСЕ блоки второй матрицы». Неподвижная матрица — B.

### Что изменилось по сравнению с C1

| | C1 | C2 |
|---|---|---|
| Сколько раз B читается из памяти | 384 раза | 1 раз — каждая подматрица один раз |
| Спуск по столбцу B | 384 блока, из памяти | 48 блоков, из L2 |
| Строка A | 307 КБ, в L1 не помещается | 48 блоков, 38,4 КБ, живут в L1 |
| Загрузок и сохранений блока C | 384 на блок | 8 на блок — раз на ряд подматриц |

### Регистровый уровень: результат не покидает регистры

Ядро лабораторной №1 держит в регистрах одну строку блока результата и только на время одного произведения блоков. В C2 в регистрах пять строк блока результата, и они остаются там на всю глубину подматрицы — 48 произведений блоков подряд.

```
for (inner_k = 0; inner_k < 10; ++inner_k) {
    right_0..right_4 = строка inner_k блока B;          /* 5 загрузок */
    for (row = 0; row < 5; ++row) {
        left = broadcast(A[row][inner_k]);              /* 1 загрузка */
        acc[row][0..4] = accumulate_product(acc[row][0..4], left, right_0..4);
    }
}
```

На шаг — десять загрузок на 25 операций вместо шести на пять у ядра лабораторной №1. 25 аккумуляторов, пять векторов строки B и один broadcast — 31 из 32 векторных регистров NEON; `objdump` не показывает ни одного сброса векторного регистра на стек в цикле. Внутри каждого элемента *k* по-прежнему идёт по возрастанию.

### Без копирования и без транспонирования

Блоки B читаются прямо из матрицы: каждый блок — 800 непрерывных байт. Упаковку B в отдельный буфер проверяли на предыдущей версии ядра — выигрыша в пределах шума не было. Ничего не копируется и не переставляется, поэтому вопрос о транспонировании не возникает вовсе.

## 6 Побитовая идентичность

Сложение с плавающей точкой неассоциативно: $(a + b) + c \neq a + (b + c)$. Версии обходят блоки в разном порядке, а результат обязан совпасть до бита. Совпадение держится на двух вещах.

### 1. Одинаковый порядок слагаемых внутри каждого элемента

Важно не то, в каком порядке заполняются *разные* элементы C, а в каком порядке к *одному* элементу прибавляются произведения. Во всех версиях блоки по глубине идут по возрастанию, внутри блока *k* — тоже по возрастанию:

| ВЕРСИЯ | ПОРЯДОК ЦИКЛОВ | K ДЛЯ ОДНОГО ЭЛЕМЕНТА C |
|---|---|---|
| C0 | block_row → block_col → block_k → строка → k → столбец | 0, 1, …, 3839 |
| C1 | block_row → block_col → block_k → строка → k | 0, 1, …, 3839 |
| C2 | first_k → first_col → block_row → block_col → половина → block_k → k | 0, 1, …, 3839 |

Подматрицы меняют, какой элемент продвигается следующим, но не порядок слагаемых внутри элемента: подматрицы по глубине идут по возрастанию во внешнем цикле. Drepper в §6.2.1 переупорядочивает суммирование свободнее и оговаривает, что игнорирует эффекты округления; здесь их игнорировать нельзя, поэтому порядок по *k* сохранён везде.

### 2. Одна и та же операция

Векторные ядра выполняют каждый шаг через одну функцию `accumulate_product`, скалярное — через её однодорожечный двойник: по умолчанию раздельные умножение и сложение, как в лабораторной №1. `#pragma STDC FP_CONTRACT OFF` и `-ffp-contract=off` запрещают компилятору сливать их по своей инициативе. Сборка `make FMA=1` переключает все три ядра на слитую операцию одновременно (`fmla.2d` в векторных, `fmadd` в скалярном).

### Почему данные случайные

Программа заполняет A и B случайными числами из [−1, 1) с полными 53-битными мантиссами. Округление происходит почти на каждой операции, поэтому совпадение до бита возможно, только если порядок и операции действительно одинаковы. На маленьких целых числах проверка ничего бы не доказывала: их суммы точны в любом порядке. Это проверено напрямую — умножение 640 × 640 с суммированием по *k* в обратном порядке:

| ДАННЫЕ | РАСХОДЯЩИХСЯ ЭЛЕМЕНТОВ | НАИБОЛЬШАЯ РАЗНИЦА | MEMCMP |
|---|---|---|---|
| целые из [−8, 8] | 0 из 409 600 | 0 | РАВНЫ |
| **случайные из [−1, 1)** | **391 956 из 409 600 (95,7 %)** | **$9{,}6 \cdot 10^{-14}$** | РАЗЛИЧАЮТСЯ |

**ЧТО ЭТО ДОКАЗЫВАЕТ**

На целых данных ядро с другим порядком суммирования прошло бы проверку, на случайных — нет. Программа сверяет C2 и C0 с C1 на случайных данных, в обоих режимах арифметики, и обе совпадают побитово. Три независимо написанных ядра — скалярное, векторное и блочное — дают один и тот же результат до бита; это и проверка правильности самого умножения.

## 7 Результаты измерений

384 × 384 блока по 10 × 10 `double` (3840 × 3840, 112,5 МиБ на матрицу), $1{,}13 \cdot 10^{11}$ операций с плавающей точкой, один поток. Сборка `-O3 -march=native`, лучший из трёх прогонов, раздельные умножение и сложение — как в лабораторной №1. `make run` идёт около 50 секунд.

```
=== verification (memcmp, no epsilon) ===
C2 equals C1, bit-exact              : PASS
C0 equals C1, bit-exact              : PASS

=== results ===
version                       time, ms    counter ticks    GFLOP/s    vs C0    vs C1
C0 (scalar)                   8606.803       8606805691     13.158    1.00x    0.66x
C1 (vectorized, lab 1)        5719.373       5719375370     19.800    1.50x    1.00x
C2 (cache-optimised)          3397.724       3397726619     33.330    2.53x    1.68x
```

| ПО ПЯТИ ПРОГОНАМ | РАЗДЕЛЬНО (КАК ЛАБ. №1) | FMA (`MAKE FMA=1`) |
|---|---|---|
| C0, GFLOP/s | 12,8–13,7 | 20,2 |
| C1, GFLOP/s | 17,9–21,4 | 26,7 |
| C2, GFLOP/s | 30,4–33,3 | 57,2 |
| C1 / C0 — векторизация | 1,35–1,59× | 1,32× |
| **C2 / C1 — оптимизация под кэш** | **1,56–1,71×** | **2,14×** |
| **C2 / C0 — обе лабораторные** | **2,32–2,61×** | **2,83×** |

Сильнее всего от прогона к прогону плавает C1 — 17,9–21,4 GFLOP/s: она часть времени ждёт память, и её скорость зависит от состояния машины. C2 стабильнее — 30,4–33,3.

Счётчик `cntvct_el0` — аналог `rdtsc` на ARM — тикает с фиксированной частотой 1 ГГц, а не с частотой ядра, поэтому его значения названы тиками.

**Проверки memcmp**

`PASS` — C2 и C0 совпадают с C1 по всем 14 745 600 элементам, на случайных данных, в обоих режимах

**make disasm**

C0 — 100 `fmul` и 100 `fadd` по скалярным `d`-регистрам, ни одной векторной; C1 и C2 — только `fmul.2d` и `fadd.2d`; с `FMA=1` — только слитые

# 8 Анализ

## Вклад регистров и подматриц

Если задать подматрицу размером со всю матрицу — `LAB2_DEPTH=384 LAB2_WIDTH=384 make run`, — C2 работает без кэш-блокировки: те же 25 регистров, тот же порядок, но спуск по столбцу B идёт по всем 384 блокам из памяти. Так вклад двух частей C2 измеряется по отдельности:

| ВЕРСИЯ, GFLOP/S | РАЗДЕЛЬНО | FMA |
|---|---|---|
| C1 | 20,3–21,0 | 26,7 |
| C2 без подматриц — только регистры | 25,1–28,1 | 37,8 |
| **C2 — регистры и подматрицы** | **33,2–33,3** | **57,2–58,9** |
| вклад регистров | ≈ 1,24–1,34× | ≈ 1,41× |
| вклад подматриц | ≈ 1,19–1,32× | ≈ 1,56× |

- **Регистры** убирают лишние загрузки: блок C больше не гоняется через память на каждое произведение блоков, а на каждую загрузку приходится 2,5 операции вместо 0,8.
- **Подматрицы** убирают ожидание памяти: спуск по столбцу B попадает в L2 (2,7–8 нс по `make probe`), а не в память (около 86 нс вразброс), и каждый кусок B читается из памяти один раз. Трафик к памяти падает с 45,7 ГБ до примерно 2,4 ГБ: B — один раз, A — 3 раза (по разу на столбец подматриц), C — 8 раз на чтение и запись (по разу на ряд по глубине).

## Почему не больше

С раздельными умножением и сложением C2 упирается в потолок арифметики одного ядра — около 33 GFLOP/s. Признаки этого:

- **Форма подматрицы не важна:** от 48 × 48 до 96 × 96 блоков скорость одинакова в пределах 3 %. Если бы ограничивала память, размер имел бы значение.
- **С FMA потолок выше** — одна инструкция вместо двух, — и те же регистры и подматрицы дают уже 2,14–2,2× к C1.

## Сравнение с лабораторной №1

Вывод программы лабораторной №1 на той же машине — матрица 400 × 400 блоков (4000 × 4000). Размеры у лабораторных разные, но GFLOP/s сравнимы:

```
version                  time, ms       counter ticks      GFLOP/s    speedup
scalar (no vector)       7490.708          7490708347       17.088      1.00x
auto-vectorized C1       4175.592          4175591852       30.654      1.79x
sse2 intrinsics C2       4108.508          4108508033       31.155      1.82x
```

| GFLOP/S | ЛАБ. №1, БЛОКИ I → K → J | ЛАБ. №2, БЛОКИ I → J → K |
|---|---|---|
| Скалярная версия | 17,1 | 12,8–13,7 (C0) |
| Векторная, то же ядро SSE2 | 31,2 | 17,9–21,4 (C1) |
| **Векторизация** | **1,82×** | **1,35–1,59×** |
| Оптимизация под кэш лаб. №2 | – | 30,4–33,3 (C2) |

### Почему векторизация здесь даёт меньше

Предел для `double` в 128-битном регистре — 2×. В лабораторной №1 обе версии шли в порядке i → k → j: B читается подряд, ждать почти нечего, и векторизация дала 1,82×. В лабораторной №2 C0 и C1 идут в порядке i → j → k и часть времени ждут блоки B. Векторизация вдвое сокращает арифметику, но не ожидание. Если считать, что арифметика сокращается ровно вдвое, прогон из §7 раскладывается так:

| ВЕРСИЯ | ВСЕГО | АРИФМЕТИКА | ОЖИДАНИЕ ПАМЯТИ |
|---|---|---|---|
| C0, скалярная | 8,61 с | ≈ 5,8 с | ≈ 2,8 с |
| C1, векторная | 5,72 с | ≈ 2,9 с | ≈ 2,8 с |

### C2 и программа лабораторной №1

Без FMA C2 (30,4–33,3 GFLOP/s) работает на уровне SSE2-версии лабораторной №1 (31,2): 0,98–1,07×. Обе упираются в один потолок арифметики, но приходят к нему разными путями. Лабораторная №1 — порядком i → k → j: B читается подряд, и память подаёт её через предвыборку. C2 — подматрицами и регистрами: B читается с прыжками, но из кэша. С FMA, где потолок выше, путь C2 выигрывает: 57–59 против 42,6 GFLOP/s у того же ядра в порядке лабораторной №1, в 1,34–1,38 раза.

**ЕСЛИ ВЫВОДЫ ДВУХ ЛАБОРАТОРНЫХ ПОЛОЖАТ РЯДОМ**

Ядро C1 — то же, что в лабораторной №1. Медленнее оно из-за порядка блоков: задание требует версию только с векторизацией, а i → k → j — уже оптимизация под кэш. Без FMA C2 на уровне программы лабораторной №1: на одном ядре M5 обе упираются в арифметику. С FMA C2 быстрее порядка лабораторной №1 в 1,34–1,38 раза.

## Что сработало на каждом уровне

| УРОВЕНЬ | ОПТИМИЗАЦИЯ | ЭФФЕКТ НА M5 |
|---|---|---|
| Кэш-линии | матрица матриц из лаб. №1: блок — 800 непрерывных байт, линия используется целиком | заложено в C1 |
| Регистры | 5 строк блока результата на всю глубину подматрицы | ≈ 1,24–1,34× (FMA 1,41×) |
| **L2 и L1d** | **подматрицы по формуле задания на обоих уровнях** | **≈ 1,19–1,32× (FMA 1,56×)** |

На x86 картина другая. Там L2 — сотни килобайт, а пропускная способность памяти на ядро заметно ниже, поэтому прокачка 45 ГБ стала бы узким местом, и сокращение трафика почти в 20 раз напрямую превратилось бы во время.

# 9 Вопросы на защиту

**Почему матрица матриц сохранена?**

В задании строка «матрицу матриц можно удалить» зачёркнута. Блочная структура лабораторной №1 сохранена полностью, а оптимизация под кэш строится поверх неё — по индексам блоков.

**Что такое подматрица и чем она отличается от блока?**

Блок — матрица 10 × 10 из лабораторной №1, элемент матрицы матриц. Подматрица — прямоугольник из целых блоков, который держится в кэше: подматрица B — 48 × 128 блоков, 4,9 МБ. В формуле задания она названа блоком; здесь это слово занято.

**В лабораторной №1 был порядок i → k → j, почему у C1 i → j → k?**

Задание требует версию только с векторизацией, а i → k → j — уже оптимизация под кэш: при нём B читается подряд. Поэтому в C1 ядро то же, а порядок исходный. C2 решает ту же проблему другим способом — подматрицами в кэше.

**Почему C1 медленнее, чем SSE2-версия в выводе лабораторной №1?**

Ядро то же, другой порядок блоков: 17,9–21,4 GFLOP/s в i → j → k против 31,2 в i → k → j. В порядке i → j → k соседние блоки B лежат на разных страницах, и ядро часть времени ждёт память.

**Насколько C2 быстрее самой программы лабораторной №1?**

Без FMA — на её уровне, 0,98–1,07×: на одном ядре обе упираются в арифметику, только разными путями. С FMA C2 быстрее того же ядра в порядке лабораторной №1 в 1,34–1,38 раза.

**Откуда выигрыш C2 над C1?**

Из двух частей, измеренных отдельно: регистры — около 1,24–1,34×, подматрицы — около 1,19–1,32× (с FMA 1,41× и 1,56×). C2 без подматриц запускается командой `LAB2_DEPTH=384 LAB2_WIDTH=384 make run`.

**Почему векторизация здесь 1,35–1,59×, а в лабораторной №1 — 1,82×?**

В порядке i → j → k обе версии часть времени ждут блоки B. Векторизация вдвое сокращает арифметику (≈ 5,8 → 2,9 с), но не ожидание памяти (≈ 2,8 с у обеих), поэтому общий выигрыш меньше.

**Зачем C0, если в задании её нет?**

Чтобы разделить вклад двух лабораторных: C1 / C0 — векторизация (1,35–1,59×), C2 / C1 — оптимизация под кэш (1,56–1,71×), C2 / C0 — обе вместе (2,32–2,61×). C0 — то же простое ядро лабораторной №1, собранное без векторизации, в том же порядке, что C1.

**Откуда формула и где у вас L3?**

В уровне кэша должны уместиться три подматрицы — от A, B и C, — с запасом 10 %. L3 у Apple silicon нет; его роль играет L2 производительного кластера, 16 МБ. Формула применена дважды: к L2 (ширина подматрицы B, 128 блоков) и к L1d (глубина, 48 блоков).

**Какая подматрица остаётся неизменной?**

Подматрица B, 48 × 128 блоков. Пока мимо неё не пройдут все 384 строки блоков A, программа не переходит к следующей.

**Как выполнена кратность ширины 64 байтам в блочной матрице?**

Ширины подматриц кратны 8 блокам: 640 байт ширины как матрица и 6400 подряд идущих байт в памяти — оба числа кратны и 64, и 128, линии кэша M5. Ширины — 48 блоков (3840 байт) и 128 блоков (10 240 байт).

**В чём преимущество оптимизации под кэш — в цифрах?**

По времени: 1,56–1,71× к C1, с FMA — 2,14×. По трафику к памяти: почти в 20 раз меньше, около 45,7 → 2,4 ГБ. Цена промаха на этой машине измерена: загрузка из памяти в 127 раз дороже, чем из L1.

**Почему C2 не быстрее 33 GFLOP/s без FMA?**

Это потолок арифметики одного ядра для раздельных умножения и сложения: форма подматрицы на скорость не влияет, а с FMA, где на операцию нужна одна инструкция, C2 сразу выходит на 57–59.

**Почему выигрыш от кэша не на порядок?**

Порядок величины получается на плоской матрице, где C1 читает B по столбцам и из каждой 128-байтной линии берёт 16 байт. Матрица матриц устранила это ещё в лабораторной №1: каждый блок — 800 непрерывных байт, линия используется целиком в любом порядке обхода.

**Почему данные случайные, а не целые?**

На целых числах суммы точны в любом порядке, и проверка ничего не доказывает: суммирование по $k$ в обратном порядке даёт на них ноль расхождений. На случайных дробных расходятся 95,7 % элементов. Программа работает на случайных данных, и все версии совпадают с C1 побитово.

**Зачем 25 аккумуляторов?**

Пять строк блока результата остаются в регистрах на всю глубину подматрицы — 48 произведений блоков подряд, — и C загружается и сохраняется один раз на подматрицу. На каждом шаге строка B, загруженная в регистры, служит пяти строкам A. 31 из 32 регистров NEON, сбросов на стек нет.

**Почему не упаковываете B, как в GotoBLAS?**

В блочной матрице каждый блок и так лежит подряд. Упаковку проверяли — выигрыша в пределах шума не было. Без копирования код проще, и ничего не переставляется.

## Источники


- U. Drepper. *What Every Programmer Should Know About Memory*, 2007. — §6.2.1: подзадачи под кэш и переиспользование загруженного; §6.3.1: аппаратная предвыборка (8–16 потоков, не пересекает границ страниц).
- K. Goto, R. van de Geijn. *Anatomy of High-Performance Matrix Multiplication*. — многоуровневая блокировка, результат в регистрах.
- A. Krivutsenko. *GotoBLAS — Anatomy of a Fast Matrix Multiplication*, TU München, 2008.
- Intel. *AP-930: Streaming SIMD Extensions — Matrix Multiplication*, 1999. — схема broadcast-and-accumulate из лабораторной №1.